I3 · Joindre des tableaux

~3 h

À la fin de cette séance vous saurez

- expliquer ce qu'est une **clé** et vérifier qu'elle est unique ;
- combiner des tableaux avec `left_join()`, `inner_join()`, `full_join()` et `merge()` de R de base ;
- trouver les lignes qui **ne** correspondent **pas** avec `anti_join()` ;
- joindre sur **plusieurs clés** (zone + année + mois) et vérifier le résultat ;
- construire le tableau d'analyse unique utilisé dans les séances suivantes.

⇩ checkpoint: I1_lines_dated.csv **⇩ checkpoint: I0_farmers.csv**

⇩ checkpoint: I2_water_quality_monthly.csv

Pourquoi plusieurs tableaux ?

Les bonnes bases de données de terrain évitent de répéter l'information. Le technicien n'a pas saisi le village, le sexe et la technique du producteur sur chacune des 2 575 lignes : il les a saisis **une fois** dans la feuille `farmers`, et chaque ligne ne porte que le `farmer_id`. Cette colonne est une **clé** : elle relie les deux tableaux. Pour analyser la croissance par sexe ou par technique, il faut amener l'information du producteur à côté de chaque ligne — une **jointure**.

~ **Contexte simulé de `villages.csv`.** À la création de la coopérative, le technicien a relevé au GPS la position (longitude, latitude, WGS84) de la plage de débarquement de chacun des 12 villages, et attribué chaque village à une **zone** (Nord, Centre, Sud) utilisée par l'ONG pour les stations de qualité de l'eau. **Type de données :** table de référence (une ligne par village), coordonnées spatiales.

Les clés doivent être uniques d'un côté

Avant de joindre, vérifiez que la clé identifie **une seule ligne** dans la table de référence. Sinon chaque ligne serait copiée plusieurs fois.

```r
library(dplyr)
sum(duplicated(farmers$farmer_id))        # 0: one row per farmer
```

```
#> [1] 0
```

```r
all(lines$farmer_id %in% farmers$farmer_id)   # every line has a known farmer
```

```
#> [1] TRUE
```

Les quatre jointures principales

Fonction	Garde	Usage typique
`left_join(x, y)`	toutes les lignes de `x`, avec `y` ajouté quand la clé correspond (`NA` sinon)	ajouter de l'information à votre tableau principal — **la plus utilisée**
`inner_join(x, y)`	seulement les lignes dont la clé est dans les deux	analyser seulement les cas complets
`full_join(x, y)`	toutes les lignes des deux	comparer deux listes
`anti_join(x, y)`	les lignes de `x` **sans** correspondance dans `y`	trouver les erreurs : identifiants inconnus, enregistrements manquants

```r
lf <- left_join(lines, farmers, by = "farmer_id")
dim(lines); dim(lf)                       # same number of rows, more columns
```

```
#> [1] 2575   15
```

```
#> [1] 2575   21
```

```r
lf[1:3, c("line_id", "farmer_id", "village", "sex", "technique", "sgr")]
```

```
#>   line_id farmer_id  village sex technique      sgr
#> 1   L0001      F013 Ankarabe   M long-line 4.187869
#> 2   L0002      F013 Ankarabe   M long-line 3.592181
#> 3   L0003      F013 Ankarabe   M long-line 3.733980
```

R BASE **DPLYR**

```r
lf2 <- merge(lines, farmers, by = "farmer_id",
             all.x = TRUE)   # all.x = left join
dim(lf2)
```

```
#> [1] 2575   21
```

```r
lf3 <- left_join(lines, farmers,
                 by = "farmer_id")
dim(lf3)
```

```
#> [1] 2575   21
```

`merge()` trie le résultat par la clé ; `left_join()` garde l'ordre d'origine. `all = TRUE` donne une jointure complète, `all.y = TRUE` une jointure à droite, rien une jointure interne.

Trouver les problèmes avec anti_join()

L'enquête (séance I4) a un tableau des ménages et un tableau des membres reliés par `hh_id`. Les membres dont le ménage n'existe pas révèlent des erreurs de saisie :

```r
hh <- read.csv("data/survey_households.csv")
members <- read.csv("data/survey_members.csv")
anti_join(members, hh, by = "hh_id")
```

```
#>   hh_id member_no relation sex age in_school works_seaweed
#> 1 HH901         1     head   F  63                      no
#> 2 HH903         2    other   M  59                      no
#> 3 HH904         3    child   M  22                      no
#> 4 HH902         3    other   M  18                      no
```

> ! **Une jointure qui ajoute des lignes est un signal d'alerte.** Si `hh` contient un ménage en double (entretien saisi deux fois), chaque membre de ce ménage sera copié deux fois après la jointure. Comparez toujours `nrow()` avant et après, et vérifiez `sum(duplicated(cle))` sur la table de référence.

```r
sum(duplicated(hh$hh_id))
```

```
#> [1] 5
```

```r
nrow(members); nrow(left_join(members, hh, by = "hh_id"))
```

```
#> [1] 2348
```

```
#> Warning in left_join(members, hh, by = "hh_id"): Detected an unexpected many-to
-many relationship between `x` and `y`.
#> i Row 544 of `x` matches multiple rows in `y`.
#> i Row 1 of `y` matches multiple rows in `x`.
#> i If a many-to-many relationship is expected, set `relationship =
#>   "many-to-many"` to silence this warning.
```

```
#> [1] 2376
```

Joindre sur plusieurs clés

Pour relier la croissance à la température de l'eau, il faut la température moyenne **de la zone du village, le mois du semis**. Le résumé de qualité de l'eau de la séance I2 a une ligne par `zone` , `year` et `month` : la jointure utilise trois clés à la fois.

```
villages <- read.csv("data/villages.csv")
wq_month <- cp_wq_monthly()                  # = I2_water_quality_monthly.csv
head(wq_month, 3)
```

```
#>     zone year month temp_mean sal_mean
#> 1 Centre 2023     1  27.85000  33.8250
#> 2 Centre 2023     2  29.12857  33.5625
#> 3 Centre 2023     3  28.32500  33.6625
```

```
seaweed <- lines %>%
  left_join(farmers, by = "farmer_id") %>%
  left_join(villages[, c("village", "zone")], by = "village") %>%
  left_join(wq_month[, c("zone", "year", "month", "temp_mean")],
            by = c("zone", "year", "month"))
dim(seaweed)
```

```
#> [1] 2575   23
```

```
sum(is.na(seaweed$temp_mean))                # 0: every line found its temperatur
e
```

```
#> [1] 0
```

ℹ Quand la clé a des **noms différents** dans les deux tableaux : `by = c("village" = "village_name")` . Dans `merge()` : `by.x = "village", by.y = "village_name"` .

⇩ **checkpoint: I3_seaweed_analysis.csv**

Travail d'application et quiz

Travail d'application I3

Script `scripts/i03_joins.R` .

Partie A — tableau d'analyse des algues

1. Joignez les lignes (I1) avec les producteurs (I0), les villages (`zone`) et les températures mensuelles (I2) comme ci-dessus. Combien de colonnes compte le tableau final ?
2. SGR moyen des lignes cultivées par des **femmes** (3 décimales) ?
3. SGR moyen des lignes en **long-line** (3 décimales) ?
4. Quelle zone a le SGR moyen le plus élevé ?
5. `temp_mean` moyenne des lignes de **Lovobe** (3 décimales) ?

Partie B — tableaux de l'enquête

6. Combien de lignes de membres ont un `hh_id` qui n'existe pas dans le tableau des ménages ?
7. En joignant les membres avec le tableau brut des ménages : combien de lignes sont **ajoutées** (à cause des ménages en double) ?
8. Retirez les ménages en double, puis `inner_join()` membres et ménages. Combien de lignes ?
9. Combien de ménages ne possèdent **aucun** bien (`anti_join` avec `survey_assets.csv`) ?

Question 7 : comparez `nrow(left_join(members, hh, by = "hh_id"))` avec `nrow(members)` .
Question 9 : `anti_join(households, assets, by = "hh_id")` garde les ménages absents du tableau des biens.

Quiz : répondez en ligne sur le site du cours (votre score y est enregistré).

by Aroo Falinirina

by Aroo Falinirina